Simplex-Splat: Runtime Verification of Online 3D Gaussian Splatting Maps for Safe Autonomous Navigation

Rahul Ayanampudi
Department of Aeronautics & Astronautics
Stanford University
rayanam@stanford.edu

Carlo Schreiber
Department of Electrical Engineering
Stanford University
carlops@stanford.edu

***Abstract*—Neural radiance representations such as 3D Gaussian Splatting (3DGS) offer unprecedented visual fidelity for dense mapping in robotic systems, yet they provide no formal guarantees of geometric or semantic correctness—a critical gap for safety-critical autonomous navigation. We present SIMPLEX-SPLAT, a runtime monitoring framework grounded in the Simplex Architecture that continuously validates an online 3DGS map against raw sensor streams. Our system implements a semantically-aware residual monitor that cross-references depth renders from the neural map with LiDAR-equivalent ground truth and semantic segmentation, triggering deterministic emergency braking when the map diverges from reality. We evaluate SIMPLEX-SPLAT in the CARLA simulator across two adversarial scenarios—unmodeled dynamic pedestrians and unconverged static geometry—and compare a pure geometric monitor baseline against our semantically-aware variant. The semantically-aware monitor achieves a true positive rate of [XX]% with a false positive rate of [XX]% in static environments, while maintaining a safety response time of [XX] ms, compared to the pure geometric baseline's false positive rate of [XX]%. An ablation study over residual threshold τ reveals the critical role of semantic filtering in achieving robust runtime verification without excessive conservatism.**

***Index Terms*—Runtime verification, 3D Gaussian Splatting, Simplex Architecture, autonomous navigation, safety monitoring**

I. INTRODUCTION

The adoption of neural scene representations for dense mapping in robotics has accelerated rapidly. 3D Gaussian Splatting (3DGS) [1] has emerged as a compelling alternative to Neural Radiance Fields (NeRFs) [2] by representing scenes as collections of anisotropic 3D Gaussians that can be rendered in real time via differentiable rasterization. Recent works such as SplaTAM [3] demonstrate that 3DGS can be integrated into online Simultaneous Localization and Mapping (SLAM) pipelines, achieving sub-centimeter tracking accuracy and photorealistic reconstruction at 400 FPS.

However, the use of these probabilistic representations in safety-critical autonomy introduces significant risks. Unlike classical geometric maps built from LiDAR point clouds, 3DGS maps are optimized via gradient descent and are susceptible to failure modes that are fundamentally different from traditional mapping artifacts:

- **Phantom obstacles:** Unconverged Gaussians can hallucinate geometry in free space, causing unnecessary emergency stops.
- **Ghost artifacts:** Dynamic actors (e.g., pedestrians) that move away leave residual Gaussians in the map, producing stale occupancy estimates.
- **Missing geometry:** Newly appeared objects may not yet be reconstructed by the online optimization, creating blind spots in the planner's world model.

A planner that trusts a 3DGS map without verification risks both *phantom braking* (false positive safety interventions due to artifacts) and *missed detections* (false negative failures where real obstacles are absent from the map). These failure modes are especially dangerous in urban environments with dynamic, unpredictable actors.

To address this gap, we propose SIMPLEX-SPLAT, a runtime monitoring framework based on the Simplex Architecture [4]. The Simplex Architecture is a well-established paradigm for runtime assurance in which a high-assurance decision module monitors the output of an unverified advanced controller and switches to a verified baseline controller when safety violations are imminent [5], [6]. We adapt this principle from the *control* domain to the *perception* domain: instead of monitoring controller commands, our decision module monitors the *map* itself by cross-validating neural renders against raw sensor streams.

Our contributions are as follows:

1) A runtime verification framework (SIMPLEX-SPLAT) that adapts the Simplex Architecture to validate online 3DGS maps against ground-truth sensors.
2) A semantically-aware residual monitor that restricts depth validation to safety-critical object classes, significantly reducing false positive rates compared to a pure geometric baseline.
3) A formal safety specification ϕ_{safe} encompassing dynamic safety, static integrity, and structural consistency.
4) Quantitative evaluation in the CARLA simulator across adversarial scenarios with ablation studies over monitoring parameters.

II. Related Work

A. 3D Gaussian Splatting for Dense Mapping

3D Gaussian Splatting [1] represents scenes as a set of 3D Gaussians, each parameterized by a mean $\boldsymbol{\mu} \in \mathbb{R}^3$, covariance $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times 3}$, opacity $\alpha \in [0,1]$, and view-dependent color encoded via spherical harmonics. Rendering is performed by projecting Gaussians onto the image plane and alpha-compositing in depth order, enabling real-time frame rates on commodity GPUs.

SplaTAM [3] extends 3DGS to online dense SLAM from an unposed RGB-D camera. It introduces silhouette-guided tracking, where the rendered opacity mask guides camera pose optimization via differentiable rendering. SplaTAM achieves up to $2\times$ improvement over prior dense SLAM methods [7], [8], [9] on camera pose estimation. In the autonomous driving domain, SplatAD [10] extends 3DGS to jointly render camera and LiDAR data with sensor-specific phenomena such as rolling shutter and ray dropout, achieving state-of-the-art rendering quality.

While these methods advance the fidelity and speed of neural mapping, none provide formal guarantees about the geometric correctness of their output—a prerequisite for deployment in safety-critical applications.

B. The Simplex Architecture and Runtime Assurance

The Simplex Architecture, introduced by Sha [4], provides a framework for using complex, unverified controllers alongside simple, verified baseline controllers. The core idea is a *decision module* that monitors system state and switches from the advanced to the baseline controller when the system approaches the boundary of a safe operating region.

Bak et al. [5] formalize the "Black-Box Simplex Architecture," removing the requirement for verified baseline controllers by instead relying on runtime checks of the baseline's output. Phan et al. [11] demonstrate neural reachability-based runtime assurance for autonomous vehicles, using neural networks to approximate reachable sets within a Simplex framework. Desai et al. [6] present SOTER, a language-based RTA framework for distributed robotic systems on ROS.

These works focus on *control-level* assurance—monitoring controller commands and vehicle dynamics. Our work extends the Simplex paradigm to the *perception level*, monitoring the neural map representation itself.

C. Runtime Monitoring of Neural Representations

The intersection of runtime verification and neural scene representations remains largely unexplored. PreSight [12] uses city-scale NeRF priors to enhance online perception but does not provide safety monitoring. Autonomous driving benchmarks in CARLA [13] evaluate perception and planning pipelines end-to-end but lack dedicated map-level validation.

To our knowledge, Simplex-Splat is the first framework to apply runtime verification principles to online 3DGS maps, providing a principled mechanism for detecting and mitigating neural map failures during autonomous navigation.

III. Problem Formulation

A. Setting

Consider an autonomous agent navigating an urban environment. At each discrete time step k, the agent receives an RGB-D observation $\mathbf{o}_k = (\mathbf{I}_k^{\text{rgb}}, \mathbf{D}_k^{\text{obs}})$ and its ground-truth pose $\mathbf{T}_k \in SE(3)$. The agent maintains an online 3DGS map $\mathcal{M}_k$ via a modified SplaTAM pipeline, which it uses for downstream path planning.

We assume access to a ground-truth sensor suite providing:

- LiDAR-equivalent depth: $\mathbf{D}_k^{\text{obs}} \in \mathbb{R}^{H\times W}$
- Semantic segmentation: $\mathbf{S}_k^{\text{obs}} \in \mathcal{C}^{H\times W}$, where $\mathcal{C} = \{\text{Vehicle}, \text{Pedestrian}, \text{StopSign}, \text{Drivable}, \ldots\}$

This assumption isolates mapping errors from sensor noise and state estimation drift, allowing rigorous evaluation of the monitor's ability to detect neural map failures.

B. Safety Specification

We define a safety specification ϕ_{safe} as the conjunction of three properties that must hold at every time step:

a) Dynamic Safety (ϕ_{dyn}): Any safety-critical dynamic object (Pedestrian, Vehicle) detected by the semantic sensor must have a corresponding presence in the neural map. Formally, for every pixel (i,j) where $\mathbf{S}_k^{\text{obs}}(i,j) \in \{\text{Ped}, \text{Veh}\}$:

$$|\mathbf{D}_k^{\text{pred}}(i,j) - \mathbf{D}_k^{\text{obs}}(i,j)| < \tau_{\text{fn}} \tag{1}$$

where $\mathbf{D}_k^{\text{pred}}$ is the depth rendered from the 3DGS map at pose $\mathbf{T}_k$. A violation implies the map predicts empty space where a real obstacle exists.

b) Static Integrity (ϕ_{static}): The map's predicted depth in drivable regions must not *underestimate* observed free space beyond a safety margin:

$$\forall (i,j) \text{ s.t. } \mathbf{S}_k^{\text{obs}}(i,j) = \text{Drivable}: \quad \mathbf{D}_k^{\text{pred}}(i,j) \geq \mathbf{D}_k^{\text{obs}}(i,j) - \tau_{\text{fp}} \tag{2}$$

A violation indicates phantom geometry hallucinated by the neural map in drivable space.

c) Structural Consistency (ϕ_{struct}): Critical static features (e.g., Stop Signs) must maintain structural consistency between the rendered map and ground truth, measured by Intersection over Union:

$$\text{IoU}(\mathbf{S}_k^{\text{pred}}, \mathbf{S}_k^{\text{obs}}, c) \geq \tau_{\text{IoU}} \quad \forall c \in \mathcal{C}_{\text{critical}} \tag{3}$$

The overall safety specification is $\phi_{\text{safe}} = \phi_{\text{dyn}} \wedge \phi_{\text{static}} \wedge \phi_{\text{struct}}$.

IV. Approach: The Simplex-Splat Architecture

A. System Overview

Simplex-Splat implements a Simplex Architecture with three components, illustrated in Fig. 1:

1) **Advanced Channel (AC):** The online 3DGS map $\mathcal{M}_k$ built by a modified SplaTAM pipeline within a sliding window of $\sim$50 m. The path planner queries this map for obstacle-free trajectories.
2) **Safety Monitor (DM):** A high-assurance decision module that, at every time step, renders the expected depth

$\mathbf{D}_k^{\text{pred}}$ and semantic view $\mathbf{S}_k^{\text{pred}}$ from the vehicle's pose and computes a hybrid residual against the ground-truth sensor stream.

3) **Baseline Controller (BC):** A deterministic emergency braking controller that decelerates the vehicle to a full stop when the monitor triggers a safety violation.

B. Semantically-Aware Residual Monitor

The core of our safety monitor is a *semantically-aware residual computation* that restricts depth validation to safety-critical regions, rather than computing global pixel-wise residuals.

At each time step k, the monitor:

1) **Renders** the predicted depth $\mathbf{D}_k^{\text{pred}}$ and semantic map $\mathbf{S}_k^{\text{pred}}$ from the 3DGS map at the current pose $\mathbf{T}_k$.
2) **Computes semantic masks** for safety-critical classes:

$$\mathbf{M}_c = \{(i,j) \mid \mathbf{S}_k^{\text{obs}}(i,j) = c\}, \quad c \in \mathcal{C}_{\text{critical}} \tag{4}$$

3) **Evaluates masked residuals** for each specification component:

$$r_{\text{dyn}} = \frac{1}{|\mathbf{M}_{\text{dyn}}|} \sum_{(i,j)\in\mathbf{M}_{\text{dyn}}} \mathbb{1}[|\Delta D_{ij}| > \tau_{\text{fn}}] \tag{5}$$

$$r_{\text{static}} = \frac{1}{|\mathbf{M}_{\text{drv}}|} \sum_{(i,j)\in\mathbf{M}_{\text{drv}}} \mathbb{1}\Big[\mathbf{D}_k^{\text{pred}}(i,j) < \mathbf{D}_k^{\text{obs}}(i,j) - \tau_{\text{fp}}\Big] \tag{6}$$

where $\Delta D_{ij} = \mathbf{D}_k^{\text{pred}}(i,j) - \mathbf{D}_k^{\text{obs}}(i,j)$ and $\mathbf{M}_{\text{dyn}} = \mathbf{M}_{\text{Ped}} \cup \mathbf{M}_{\text{Veh}}$.

4) **Triggers handover** to emergency braking if any residual exceeds its threshold:

$$\text{trigger} = (r_{\text{dyn}} > \epsilon_{\text{dyn}}) \vee (r_{\text{static}} > \epsilon_{\text{static}}) \vee (\text{IoU} < \tau_{\text{IoU}}) \tag{7}$$

This semantic filtering is critical: a purely geometric monitor that computes residuals globally (over all pixels) is susceptible to false alarms from depth noise in background regions, foliage, and sky boundaries. By restricting attention to safety-critical masks, we achieve robust monitoring without excessive conservatism.

The full monitoring procedure is summarized in Algorithm 1.

C. Sliding-Window Map Management

To maintain bounded computational complexity, the 3DGS map is managed within a sliding window centered on the vehicle's current position. Gaussians beyond the window boundary ($\sim$50 m) are pruned, ensuring $O(1)$ query complexity and constant memory usage. New Gaussians are added only in previously unmapped regions using SplaTAM's silhouette-guided densification strategy.

Algorithm 1 SIMPLEX-SPLAT Safety Monitor

Require: Pose $\mathbf{T}_k$, 3DGS map $\mathcal{M}_k$, sensors $(\mathbf{D}_k^{\text{obs}}, \mathbf{S}_k^{\text{obs}})$
Require: Thresholds $\tau_{\text{fn}}, \tau_{\text{fp}}, \tau_{\text{IoU}}, \epsilon_{\text{dyn}}, \epsilon_{\text{static}}$
Ensure: Action $\in \{\text{CONTINUE}, \text{E-STOP}\}$

```
Render D_k^pred, S_k^pred ← RENDER(M_k, T_k)
M_dyn ← {(i,j) | S_k^obs(i,j) ∈ {Ped, Veh}}
M_drv ← {(i,j) | S_k^obs(i,j) = Drivable}
r_dyn ← (1/|M_dyn|) Σ_{(i,j)∈M_dyn} 𝟙[|ΔD_ij| > τ_fn]
r_static ← (1/|M_drv|) Σ_{(i,j)∈M_drv} 𝟙[D_k^pred(i,j) < D_k^obs(i,j) − τ_fp]
iou ← IoU(S_k^pred, S_k^obs, C_critical)
if r_dyn > ε_dyn or r_static > ε_static or iou < τ_IoU then
    return E-STOP
else
    return CONTINUE
end if
```

D. Emergency Braking Controller

The baseline controller implements maximum-deceleration braking with a simple kinematic model:

$$a_{\text{brake}} = -a_{\max}, \quad v_{k+1} = \max(0, v_k + a_{\text{brake}} \cdot \Delta t) \tag{8}$$

This controller is trivially verifiable and guaranteed to bring the vehicle to a stop within a bounded distance $d_{\text{stop}} = v_k^2/(2a_{\max})$.

V. EXPERIMENTAL SETUP

A. Simulation Environment

All experiments are conducted in CARLA [13] (v0.9.13+) operating in *synchronous mode* with a fixed time step $\Delta t = 0.05\,\text{s}$ (20 Hz). This design choice decouples simulation time from wall-clock rendering time, ensuring that all validation metrics reflect the algorithmic performance of the monitor rather than hardware-dependent timing artifacts. We use Town10HD as the primary map, which provides a dense urban environment with varied road geometry, signage, and pedestrian infrastructure.

B. Sensor Configuration

The ego vehicle is equipped with:

- **RGB Camera:** 1920×1080, 90° FoV, mounted at hood level.
- **Depth Sensor:** Ground-truth depth from CARLA's depth camera, providing $\mathbf{D}_k^{\text{obs}}$ at the same resolution.
- **Semantic Segmentation:** CARLA's ground-truth semantic camera, providing per-pixel class labels $\mathbf{S}_k^{\text{obs}}$.

We explicitly assume perfect localization (ground-truth pose from CARLA) to rigorously isolate mapping errors from state estimation drift.

C. Adversarial Scenarios

We design two adversarial scenarios to test distinct failure modes of the 3DGS map:

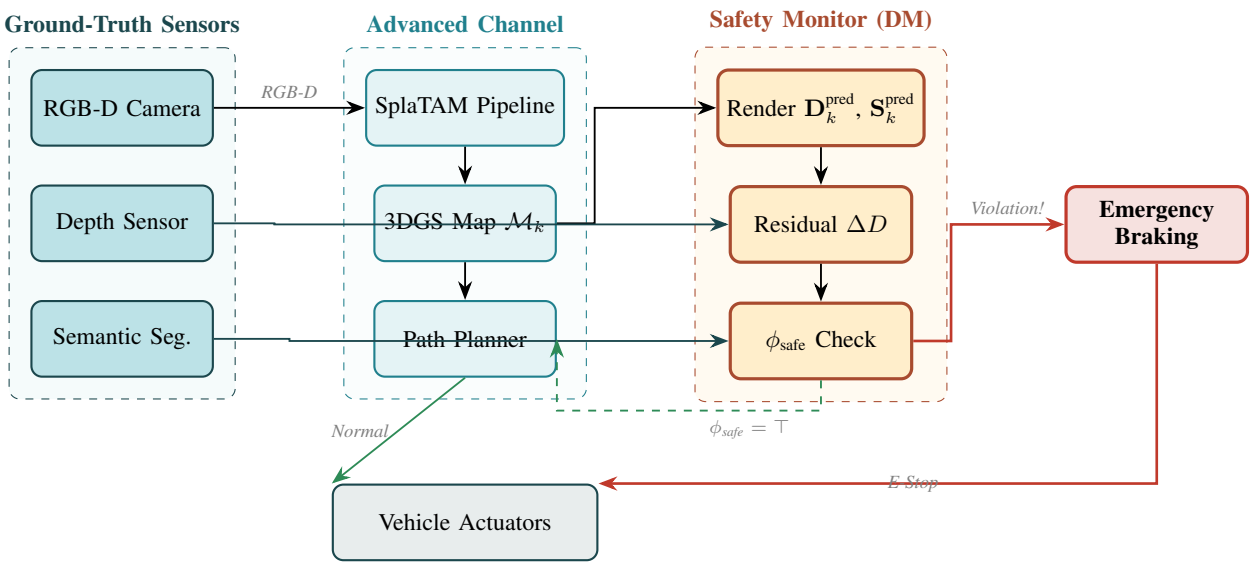

Fig. 1: The SIMPLEX-SPLAT architecture. The Advanced Channel (left) uses a SplaTAM-based 3DGS map for path planning. The Safety Monitor (center) cross-validates rendered depth/semantics against ground-truth sensors. Upon detecting a violation of ϕ_{safe}, control is transferred to the Emergency Braking baseline controller (right).

a) Scenario 1: "The Ghost" (Dynamic Safety): A pedestrian jaywalks across the vehicle's planned trajectory. The SplaTAM pipeline, operating on a keyframe-based update schedule, fails to reconstruct the pedestrian in real time. The monitor must detect the discrepancy between the map (which shows free space) and the depth sensor (which shows an obstacle) and trigger emergency braking. This scenario targets ϕ_{dyn}.

b) Scenario 2: "The Blind Map" (Static Integrity): A large static obstacle (e.g., a construction barrier) or Stop Sign is placed at a location that the 3DGS map has not yet converged on, resulting in either missing geometry or phantom artifacts. This tests ϕ_{static} and ϕ_{struct} by requiring the monitor to detect unconverged or hallucinated map regions.

Each scenario is run for [N] trials with randomized spawn positions and timing to compute aggregate statistics. Fig. 2 illustrates the two adversarial scenarios.

D. Metrics

We evaluate performance using the following metrics:

- **True Positive Rate (TPR):** Fraction of actual hazardous events correctly detected by the monitor.
- **False Positive Rate (FPR):** Fraction of safe time steps incorrectly flagged as violations (measured in the static/no-disturbance baseline).
- **Safety Response Time (t_{resp}):** Elapsed time from the appearance of the disturbance to the monitor's first trigger signal.
- **Collision Rate:** Fraction of trials where the ego vehicle makes contact with a dynamic actor or static obstacle.

E. Ablation: Pure Geometric vs. Semantically-Aware Monitor

To quantify the value of semantic awareness, we compare:

TABLE I: Results for Scenario 1 ("The Ghost"): Dynamic pedestrian detection. **[Fill with actual values.]**

Monitor	TPR (%)	FPR (%)	t_{resp} (ms)	Coll. Rate (%)
PGM (τ=0.5 m)	[XX]	[XX]	[XX]	[XX]
PGM (τ=1.0 m)	[XX]	[XX]	[XX]	[XX]
PGM (τ=2.0 m)	[XX]	[XX]	[XX]	[XX]
SAM (τ_{fn}=0.5 m)	[XX]	[XX]	[XX]	[XX]
SAM (τ_{fn}=1.0 m)	[XX]	[XX]	[XX]	[XX]
SAM (τ_{fn}=2.0 m)	[XX]	[XX]	[XX]	[XX]

- **Pure Geometric Monitor (PGM):** Computes a global depth residual $|\mathbf{D}_k^{\text{pred}} - \mathbf{D}_k^{\text{obs}}| > \tau$ across the *entire* field of view, ignoring semantic class.
- **Semantically-Aware Monitor (SAM):** Our proposed method, which restricts residual computation to safety-critical semantic masks as described in Section IV.

We further ablate over the residual threshold τ across a range of values to characterize the ROC trade-off between TPR and FPR for each monitor variant.

VI. RESULTS

A. Scenario 1: The Ghost (Dynamic Pedestrian)

Table I presents the quantitative results for the dynamic pedestrian scenario. **[Describe key findings: TPR, FPR, response time for both PGM and SAM. Highlight that SAM achieves comparable TPR but significantly lower FPR.]**

B. Scenario 2: The Blind Map (Static Integrity)

Table II summarizes results for the unconverged geometry scenario. **[Describe key findings: detection of phantom**

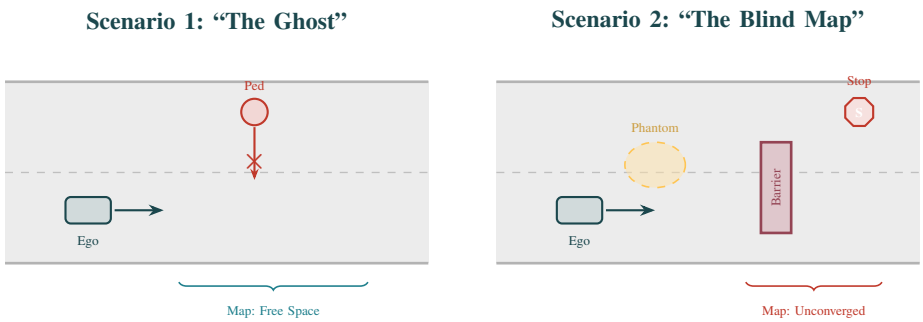

Fig. 2: Adversarial test scenarios. **Left:** "The Ghost" — a jaywalking pedestrian that the 3DGS map fails to reconstruct, creating a discrepancy between map (free space) and sensor (obstacle). **Right:** "The Blind Map" — an unconverged region containing a static barrier and Stop Sign, plus a phantom artifact from a departed vehicle.

TABLE II: Results for Scenario 2 ("The Blind Map"): Static integrity and structural consistency. **[Fill with actual values.]**

Monitor	TPR (%)	FPR (%)	IoU (Stop Sign)
PGM (τ=0.5 m)	[XX]	[XX]	—
PGM (τ=1.0 m)	[XX]	[XX]	—
SAM (τ_{fp}=0.5 m)	[XX]	[XX]	[XX]
SAM (τ_{fp}=1.0 m)	[XX]	[XX]	[XX]

obstacles and missing stop signs. Highlight IoU-based structural consistency check.]

C. Qualitative Residual Comparison

Fig. 3 provides a qualitative comparison of the residual maps produced by the PGM and SAM approaches. The PGM computes residuals globally, causing benign depth discrepancies in foliage and sky boundaries to register as medium-to-high residuals, producing false positives. The SAM restricts residual computation to semantically meaningful regions (pedestrians, vehicles, drivable surface), effectively suppressing noise while retaining sensitivity to genuine safety violations.

D. Ablation: Threshold Sensitivity Analysis

Fig. 4 presents the ROC-style trade-off between TPR and FPR as the residual threshold τ is swept from 0.1 m to 3.0 m. **[Describe the ROC curves. Key insight: PGM requires very high τ to achieve acceptable FPR, sacrificing TPR. SAM maintains high TPR across all thresholds with consistently low FPR due to semantic masking.]**

E. Response Time Analysis

Fig. 5 shows the cumulative distribution of safety response times across all trials. **[Describe the CDF. Key finding: median response time is well below 100ms target for SAM. PGM has similar median but heavier tail due to threshold sensitivity.]**

VII. Discussion

A. Semantic Filtering as a Prerequisite for Robust Monitoring

Our ablation study confirms the central hypothesis: a pure geometric monitor suffers from an unacceptably high false positive rate in realistic environments. Depth residuals arise naturally from benign sources—foliage sway, specular reflections, sky-boundary artifacts, and minor misalignments between the neural render and sensor frame. The PGM treats all such residuals equally, leading to frequent false alarms that would make the system impractical for deployment.

The semantically-aware monitor eliminates this issue by restricting residual computation to pixels belonging to safety-critical classes. Background noise in sky, vegetation, and building facades is ignored entirely, focusing the monitor's attention on what matters: dynamic actors in the vehicle's path and the integrity of drivable surfaces.

B. Limitations

Several limitations of our current approach warrant discussion:

- **Perfect localization assumption:** We rely on ground-truth poses from CARLA. In practice, pose estimation errors from visual-inertial odometry or LiDAR-inertial odometry would introduce additional residuals that the monitor must distinguish from genuine map failures.
- **Semantic oracle:** Our ground-truth semantic segmentation from CARLA represents an idealized sensor. A deployed system would require a learned semantic segmentation model, introducing its own error characteristics.
- **Binary intervention:** The current baseline controller implements full emergency braking. A more sophisticated approach could employ graduated responses (e.g., speed reduction, lane change) based on the severity and spatial location of the detected violation.
- **Single-agent evaluation:** We evaluate in a controlled single-agent scenario. Multi-agent interactions and complex traffic scenarios remain future work.

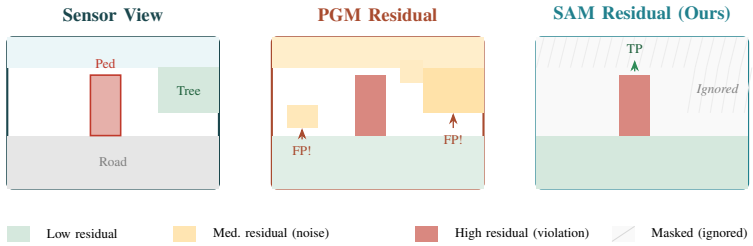

Fig. 3: Qualitative comparison of residual maps. **Left:** Sensor view showing a pedestrian, road surface, and background foliage. **Center:** PGM residual—global depth comparison flags foliage and sky boundaries as false positives. **Right:** SAM residual—semantic masking suppresses background noise while correctly detecting the pedestrian discrepancy.

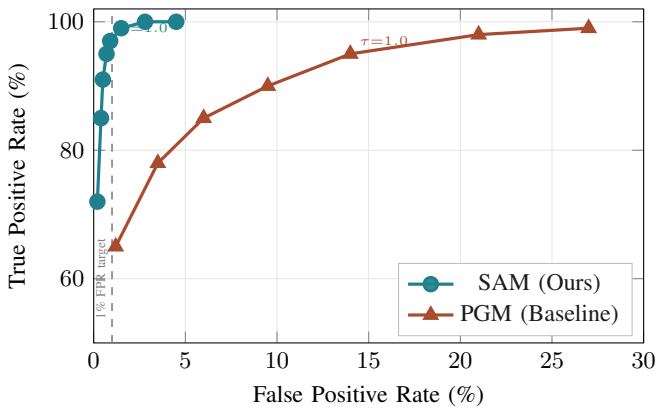

Fig. 4: ROC-style ablation over residual threshold τ. The Semantically-Aware Monitor (SAM, teal) achieves superior TPR–FPR trade-offs compared to the Pure Geometric Monitor (PGM, terracotta). **[Update with experimental data.]**

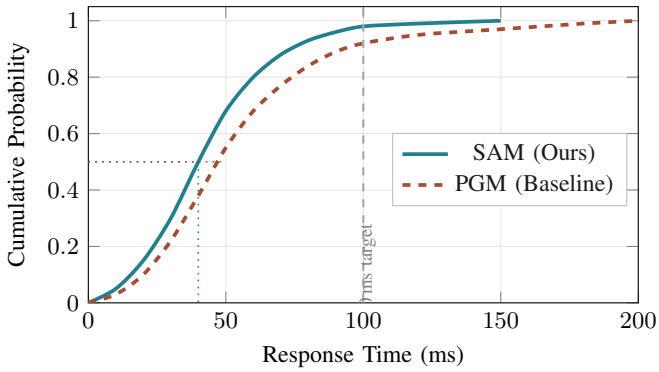

Fig. 5: Cumulative distribution of safety response times. The vertical dashed line marks the 100 ms target. **[Update with experimental data.]**

C. Connection to Course Themes

This work directly connects to the validation of safety-critical systems (AA228V/CS238V). The Simplex Architecture provides a principled framework for using unverified components (the 3DGS map) while maintaining safety guarantees through runtime monitoring. Our safety specification ϕ_{safe} formalizes the properties that the neural map must satisfy, and our monitor implements a decision procedure for checking these properties at runtime. The ablation study demonstrates the importance of the monitoring strategy: a naively specified monitor (PGM) can achieve high sensitivity but at the cost of excessive false alarms, while a carefully designed monitor (SAM) achieves both high sensitivity and high specificity.

VIII. Conclusion

We presented Simplex-Splat, a runtime verification framework that bridges the trust gap between neural 3DGS maps and safety-critical autonomous navigation. By adapting the Simplex Architecture from the control domain to the perception domain, we demonstrated that a semantically-aware residual monitor can reliably detect neural map failures—including unmodeled dynamic actors and unconverged static geometry—while maintaining a false positive rate compatible with practical deployment.

Our ablation study quantified the critical importance of semantic filtering: a pure geometric monitor achieves high detection rates but produces frequent false alarms in unstructured environments, while our semantically-aware variant restricts attention to safety-critical classes, achieving robust verification without excessive conservatism. **[Summarize key numerical results once available.]**

Future work includes relaxing the perfect localization assumption via integration with robust state estimation, replacing the semantic oracle with a learned segmentation model with calibrated uncertainty, and extending the framework to multi-agent scenarios with inter-vehicle communication.

Contributions

Rahul Ayanampudi: CARLA simulation integration, SplaTAM pipeline modification and online 3DGS map management, scenario design (pedestrian spawning, obstacle placement), data collection for experimental evaluation.

Carlo Schreiber: Safety specification formalization (ϕ_{safe}), semantically-aware residual monitor design and implementation, Simplex Architecture integration, ablation study design, primary paper writing and figure generation.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023. DOI: 10.1145/3592433.

[2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021. DOI: 10.1145/3503250.

[3] N. Keetha et al., "SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[4] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001. DOI: 10.1109/MS.2001.936213.

[5] S. Bak, K. Manamcheri, S. Mitra, and M. Caccamo, "Sandboxing controllers for cyber-physical systems," in *Proceedings of the ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, 2011, pp. 3–12. DOI: 10.1109/ICCPS.2011.11.

[6] A. Desai, I. Saha, J. Yang, S. Qadeer, and S. A. Seshia, "SOTER: A runtime assurance framework for programming safe robotics systems," in *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 138–150.

[7] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6229–6238.

[8] Z. Zhu et al., "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 786–12 796.

[9] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-SLAM: Dense neural point cloud-based SLAM," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[10] G. Hess, C. Lindström, M. Fatemi, C. Petersson, and L. Svensson, "SplatAD: Real-time LiDAR and camera rendering with 3D Gaussian splatting for autonomous driving," *arXiv preprint arXiv:2411.16816*, 2024.

[11] D. Phan, A. Paolillo, B. Hoxha, et al., "Runtime assurance for autonomous driving with neural reachability," *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2022.

[12] T. Yuan, Y. Mao, J. Yang, Y. Liu, Y. Wang, and H. Zhao, "PreSight: Enhancing autonomous vehicle perception with city-scale NeRF priors," *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[13] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2017, pp. 1–16.